only. If you are a user wanting to provide feedback, see Capturing User Feedback.

Capturing User Feedback

We highly value user feedback! Please use the epics below to capture feedback and insights for the two feature categories:

- Web IDE User Feedback & Insights
- Workspaces User Feedback & Insights

For non-team members, feel free to create issues in these epics if you have general feedback or suggestions. If you have feedback related to existing or ongoing features, please drop a comment in the appropriate epic or issue.

· Customer Collaboration Issues Dashboard

At times, it is required to create private collaboration projects under https://gitlab.com/gitlab-com/account-management to collaborate with the customers on their needs. For such issues, add the appropriate labels so that they show up in our dashboard mentioned below.

Use the comment template to apply the appropriate labels for the feature categories:

- Workspaces - /label ~"Category:Workspaces" ~"customer-collaboration"

You can find the customer collaboration issues dashboard for the feature categories:

- Workspaces

Group Metrics Dashboards

Create::Remote Development Group Metrics Tableau Workbook

· Group Meetings

··Important: For every meeting, the Remote Development group's meeting document should be used apart from High Level Planning which has a document of its own, and filled with the meeting notes, as well as references to any other sync meeting agendas/notes/recordings which have recently occurred. This will make it easier for people to find any meeting notes.

Please note that sync meeting schedules are flexible and can be moved to accommodate required participants. For up-to-date schedule of all team meetings, please consult the Group's Calendar.

The table below briefly outlines the objectives and key details of regular team meetings:

| Meeting Title | What
|

|---|---|
High Level Planning	Set overall direction and validate higher-priority issues/epics to be worked on in the upcoming releases.
Iteration Planning Meeting (IPM)	Review scheduled issues; estimate work for next milestones.
Remote Development Retro Call	Review feedback from async retro, identify action items and next steps to improve efficiency.
Engineering Sync	Discuss engineering topics and brainstorming. Cancelled if no topics. Alternates APAC/AMER friendly times.
Remote Development Pairing	Pairing sessions for engineers. Cancelled if no topics.

· Group Processes

· Weekly EM Updates

Each week the group EM provides a Weekly Status update issue which aims to capture the most important items for the team to be aware of. These can be found here.

· Issue Workflow Hygiene

In the Create:Remote Development group we leverage an automatic issue hygiene system via the triage bot. This helps to ensure issues and label hygiene are respected.

· Investigations and Breaking Down Large Issues

If a task is too large, has too many unknowns, or requires proof of concept (POC), it should be broken down into smaller investigation tasks or POC issues. These tasks help clarify the scope, reduce risks, and identify the necessary steps to proceed with implementation and ideally should fit into a single milestone.

1. Create an Investigation Issue:
   - Purpose: Research, investigate, and document or breakdown the necessary work. Please make sure to define the core question or problem you're investigating.
   - Weight: Default to 3 for investigations, POCs, or breakdown tasks. If a different weight is needed, discuss it with PM/EM/Team stakeholders.
   - Label: Assign the ~spike label to the issue.
   - Updates: Investigations are capped at 3 working days of focused effort.
     - On Day 3 or sooner, investigator shares findings and proposed next steps. Consider using a sync meeting to align with key stakeholders and make a decision. If a meeting is not feasible, a short recorded video summarizing findings is acceptable.
   - Depending on the feedback from the updates, we can decide to allocate more time to these investigations or settle for something that works based on the information we have.

1. Break Down and Close:
   - After the investigation is done, summarize findings and break the work into actionable, refined issues within the given epic.

  - The outcome should include:
    - An Architecture Plan: High-level technical direction, quality goals (like performance, security), and supporting approaches.
    - An Iteration Plan: A breakdown of work into clearly scoped, refined issues .
  - Add the plans to the epic description and close the investigation issue.

· Issue Guidelines

These guidelines apply to all issues we use for planning and scheduling work within our group. Our Engineers can define specific implementation issues when needed, but the overall goal for our issues are as follows:

- Treat the wider community as the primary audience (see relevant summary for rationale).
- Provide a meaningful title that describes a deliverable result.
  - · Add a cancel button to the edit workspace form page
  - · Automatically save Devfile changes after 2 seconds of inactivity
  - · Make WebIDE better
- Provide a meaningful description that clearly explains the goal of the issue, and provide some technical details if necessary.
- Should there be critical implementation steps or other useful ways to create small tasks as part of the issue, please use a checklist as part of the issue descriptions.
- The issue should have a weight assigned see Iteration Planning.

· Planning Process

To improve the accuracy of our planning and delivery estimates, we've adapted parts of the Plan and Build & Test phases of the GitLab Product Development Flow. Our team uses a lightweight, velocity-based approach inspired by XP and Scrum. This helps us stay flexible while still providing clear, realistic forecasts.

The goal is not to fully adopt XP or Scrum, but to take the pieces that work for us, mainly around iteration planning and historical velocity tracking. By grounding our estimates in "Yesterday's Weather" (our team's recent delivery history), we can better align scope to capacity and make informed decisions about what we can ship and when.

This process helps us navigate evolving priorities, reduce planning overhead, and stay transparent about what we're working on.

Process Phases

mermaid
graph TD;
  S[Feature Inception] -->|"New epic is created and issues added to it with Backlog assigned"| V[High Level Planning]
  V -->|"Epic is prioritized into the roadmap and on the epic board by PM adding '~(current

quarter | current quarter+1 | current quarter+2)' label "| R[Async Refinement]
  R -->|"Epic is broken down into issues and 'refined' label applied. Change epic color to 'Apricot'."| P[IPM - Sync/Async]
  P -->|"Epics marked refined have all of its issues weighed. Once all weighed, change epic color to 'Mint'."| I[Ready for Development]

  I --> N[Milestone Planning]
  I --> D[Development]

  N -->|"EM/PM will plan for the next milestone and\nassign %XX.X labels to issues that are likely to make the next release cycle."| E1[Planning Issue Published]
  D -->|"When work starts on the epic, all of its child issues\nare added to %"Next 1-3 Releases""| E2[Development Release]

1. Feature Inception

Ideas can come from anywhere and anyone. If you have an idea:

1. Capture it in an issue under the Workspaces or Web IDE User Feedback & Insights epic.
1. Pre-fix the issue title with "Feedback:..." or "Idea:...".
1. Add the issue to the %"Backlog" milestone
1. Add this as a topic of discussion on the Workspaces or Web IDE High Level Planning agenda.

2. High Level Planning

The High Level Planning meeting is an open forum where new and ongoing work is identified, discussed, and prioritized. Team members can propose topics by adding them to the agenda in advance. This is analogous to the Validation Track in the GitLab Product Flow, because it needs to achieve the same Validation Goals & Outcomes before we can start refining and prioritizing issues. The meeting typically covers:

- New Feature Ideas: Proposals for new work to be considered for the roadmap.
- Roadmap Adjustments: Reordering, shifting, or reprioritizing ongoing work.
- Escalation of Bugs/Technical Debt: Issues that need urgent attention or adjustments to the timeline.

The meeting serves to clarify the most important work and make decisions on what should take priority.

Post-Meeting Actions:

- Roadmap Assessment: After the meeting, the Product Manager will assess the proposed changes and update the epic board(s), which serves as the source of truth for work prioritization.

- Epic Creation and Prioritization: Features will be converted into epics and the Product Manager will determine the order of feature work and mark upcoming work with the ~"(current quarter | current quarter+1 | current quarter+2)" label according to the quarter that the work should start. See Fiscal Year for dates of the quarters.

- Board Order Guidelines: Please avoid changing the order of items on the epic board without consulting the Engineering Manager or Product Manager first.

3. Async Refinement

The Async Refinement process is designed to efficiently prepare upcoming work by focusing on breaking down issues and identifying unknowns in implementation. This is analogous to "backlog refinement" in the standard GitLab product development flow. The goal is to ensure all issues targeted for the upcoming milestones are clear enough for the team to briefly review and estimate during the next IPM.

Key Principles:

- Epic Board: The epic board organizes and prioritizes upcoming work, following a color scheme to reflect each epic's status.
  - Blue: Default color for new epics that need refinement.
  - Apricot: Indicates that an epic is fully refined and ready for weighing in the next planning stage.
  - Mint: Used after the Iterative Planning Meeting once all issues within an epic have been weighed and finalized for execution.

- Just-in-Time Planning: We refine only the next 1-2 epics to avoid over-preparing, which helps ensure epics remain relevant when work begins. If these are refined, no further refinement is necessary.

Refinement Process:

1. Identify Epics in Need of Refinement:
   - These are marked in blue on the epic board and would typically be assigned by the Engineering Manager to Engineers.
1. Break Down the Epic:
   - Divide the epic into smaller, actionable issues.
   - Define the work necessary to meet the epic's acceptance criteria.
   - Assign the ~refined label to the issues that have been refined.
1. Mark as Refined:
   - Once refined, change the epic color to apricot to indicate it is ready for weighing.
   - Add the label "refined" to the epic to signal readiness.
1. Next Steps - Iterative Planning Meeting:
   - Following refinement, epics enter the Iteration Planning Meeting where all issues within an epic are weighed.
   - After this stage, epics are marked mint to indicate they are fully weighed and ready for execution.

4. Iteration Planning Meeting

The Iteration Planning Meeting is a collaborative session where the team reviews and weighs issues within epics marked as apricot on the epic board. This is analogous to the "Weekly Cycle" in XP or "Sprint Planning" in Scrum. This process ensures that each refined epic is fully understood, in scope, and aligned with the team's goals.

Meeting Objectives:

- Review and Weigh Issues:
  - For each issue, the facilitator reads the description, and the team briefly discusses the issue and clarifying any uncertainties. If there are no blocking concerns/risks raised, the team collectively estimates the issue with rock-paper-scissors fibonacci scale, and the collectively agreed weight is assigned. SeeWhat Weights to Use for more details on weights.
  - If there are other prioritized issues that have not yet been weighed, these are also reviewed and weighed during the meeting.

Async Process:
TL;DR: Sometimes issues need to be weighted quickly before the official IPM meeting. This is how we weight those issues.

Prerequisite: Add the Polly app to your Slack if you have not already.

1. Navigate to Polly application under that Apps section in Slack.
1. Select Create a Polly.
1. Select Create New.
1. Select Multiple Choice
1. Fill out creation Options:
   1. Create Question: Weight for: Add link to issue here.
   1. Question Type: Multiple Choice.
   1. Enter choices below: 0 1 2 3 5 8 (each number on a separate line)
   1. Choose audience: Select remotedevelopmentasyncipm channel.
   1. Make sure "Send polly as direct message" is unchecked.
   1. Select Settings Button.
   1. Responses: Select Non-anonymous.
   1. Results: Select Show after close.
   1. Select Submit to save changes.
1. Send Polly.

Optional Steps: Template Creation

This allows you make following Async IPMs faster by standardizing the configurations. After creation, all a user needs to do is select the template from the "My Templates" section, select Use Template, and update the Issue link in the "Create Question" field.

1. Navigate to Polly application under that Apps section in Slack.
1. Select Go to Polly Dashboard.
1. Select the Polly you just made.
1. Select Controls button.
1. Select Save as Template.
1. Title Template: Remote development async ipm.
1. Ensure "Save audience with template" is checked.
1. Select Save.

Async Weighing Option:

- Weighing can also be done asynchronously through the remotedevelopmentasyncipm Slack channel.

- To initiate async weighing, post the issue that needs to be weighed along with a Polly poll to gather input.

This structure allows for both synchronous and asynchronous participation, enabling thorough preparation and alignment on upcoming work.

5. Milestone Planning & Starting Development

The Milestone Planning & Starting Development process is used to plan issues for development in upcoming releases and to align team efforts with milestone.

Epic and Issue Setup: When starting work on a new epic, all child issues are assigned the milestone %"Next 1-3 Releases" or a concrete milestone for example, %16.9 to indicate they are prioritized for near-term development.

Issues are assigned to specific milestones based on factors like team velocity, potential for parallel work, and overall availability. If unplanned work needs to be added to an active milestone, please discuss it with the EM first, as it may affect delivery projections and commitments.

Occasionally, unrefined or unplanned issues like bugs and customer escalations may be brought in after a milestone has started. In these cases, they will be labeled ~Stretch by default, since they weren't fully prepared before being included.

Issues typically move from %"Backlog" to %"Next 1-3 Releases" and then into a concrete milestone e.g. %16.x. Unless discussed otherwise with the EM, engineers should prioritize picking up ~Deliverable items scheduled in the active milestone before considering other issues in that milestone. If all ~Deliverable and ~Stretch issues in the current milestone are already assigned and in progress, the next place to look is the following milestone or %"Next 1-3 Releases".

Milestone Planning and Creating Planning Issue:

1. Before each milestone begins, the Engineering Manager along with Product Manager reviews and assigns issues for the upcoming release based on the team's velocity. Specific milestone number %XX.X to designate them as part of the planned release.

1. A Planning Issue is automatically created two weeks before start of the new release cycle. This issue is populated with relevant details to guide the team through the milestone. You can view and access all active Planning Issues here.

This structure enables smooth planning, tracking, and alignment of development work within each milestone, ensuring work progresses as planned and within scope.

Example Lifecycle for a Feature

1. Product and Design identify a feature and create an epic.
   Note that the issue description may be incomplete/unrefined and high-level at this point.
1. When prioritized, Product Manager adds the ~"(current quarter | current quarter+1 | current quarter+2)" label and the epic is assigned for refinement by the Engineering Manager.

1. As part of the async IPM process, the assignee refines the epic, by breaking down the feature work into issues and filling out the issue template, then applying the ~refined label to the issues and to the epic if all issues within the epic have been refined.
    1. During the refinement process, consider documentation for the feature. If needed, add the requirements and the ~documentation and ~Technical writing labels to the issue.
        For question and assistance, tag your assigned Technical Writer.
1. In the sync IPM meeting, the wider team discusses and estimates the issues in the epic.
1. Once the priority and weight are determined, the EM will assign a specific release milestone to the issues of the epic based on velocity.
1. The assignee opens an MR for the issue and ensures that the issue and MR are cross-referenced on the first lines of their descriptions.
1. While the feature implementation is in progress, the assignee creates a documentation MR that follows the appropriate topic type format and style guide.
1. The documentation MR is reviewed by the Technical Writer, and merged along with or shortly after the feature implementation MR.
1. Once the feature MR is merged, documentation is published, and the feature is verified in production, the epic is closed.

· Ad-Hoc Work

It is normal that team members may identify issues that need to be resolved promptly prior to the next planning cycle. This may be because they are blocking other prioritized issues, or just because a team member wishes to tackle an outstanding bug or small piece of technical debt.

In these situations, it is acceptable for a team member to take the initiative to create an issue, assign proper labels, estimate it, assign it to the current milestone, and work on it, as long as it does not adversely impact the delivery of other prioritized issues. However, if it becomes large or risks impacting other issues which were in the milestone, then the issue should be brought up for discussion with the wider team in the next IPM.

· What Weights to Use

To assign weights to issues effectively, it's important to remember that issue weight should not be tied to time. Instead, it should be a purely abstract measure of the issue's significance. To accomplish this, the team uses the Fibonacci sequence starting from weight 0:

- Weight 0: Reserved for the smallest and easiest issues, such as typos or minor formatting changes, or very minor code changes with no tests required.
- Weight 1: For simple issues with little or no uncertainty, risk or complexity. These issues may have labels like "good for new contributors" or "Hackathon - Candidate". For example:
  - Changing copy text which may be simple but take some time.
  - Making CSS or UI adjustments.
  - Minor code changes to one or two files, which require tests to be written or updated.
- Weight 2: For more involved issues which are still straightforward without much risk or complexity, but may involve touching multiple areas of the code, and updating multiple tests.
- Weight 3: For larger issues which may have some unforeseen complexity or risk, or require more extensive changes, but is still not large enough to warrant breaking down into smaller separate issues.
- Weight 5: Normally, this weight should be avoided, and indicate that the issue ideally should

be broken down into smaller separate issues. However, in some cases a issue with a weight of 5 might still be prioritized. For example, if there is a large amount of manual updates to be made which will require a large amount of effort, but doesn't necessarily involve significant risk or uncertainty.
- Weight 8/13+: Weights above 5 are used to clearly indicate work that is not yet ready to be assigned for implementation, and must be broken down because it is too large in scope to start implementing, and/or still has too many unknowns/risks. This weight is temporarily assigned to "placeholder" issues to capture the scope of the effort in our velocity-based capacity planning calculations. For more information, see "Breaking Down Large Issues".

Should bugs be estimated?

There are differing opinions on this in agile philosophy (1, 2).

On our team, we have decided that bugs should not be estimated. Here's why:

- The point of estimating weight in a velocity-based process is to help predict the rate at which a team can expect user value be delivered.
- From that perspective, bugs should not be estimated, because the "user value" was delivered by the original feature, which did have a weight.
- But fixing a bug isn't adding any new user value, it's just "finishing" delivery of the user value which was already accounted for by the original feature. So, they shouldn't get a weight.
- Now, if it's a huge "bug" in the category of "we got this feature entirely wrong and need to rewrite it significantly, and it will take a lot of effort", then that should be considered new feature work, not a "bug". And it should be refined and broken down into weighted issues, just like all feature work.

· Follow-up issues which span multiple releases

Gitlab standards often require breaking down issues that need to be resolved in a specific set of steps that span multiple releases. Typically these are issues related to database migrations (Dropping Columns) or breaking changes in GraphQL such as "Deprecation and Removal".

In such cases where we have pending or follow up tasks for future releases such as removing an ignore rule, removing a deprecated field from GraphQL, finalizing background migrations, etc., we must create follow up issues so we do not forget to complete the work in the future. Here is the process to follow:

Create a Followup Issue:

1. References: Link the issue to the original issue that spawned it.
1. Label: Assign these labels:
   - ~due-date-followup
   - ~refined
1. Milestone: Assign it a specific milestone - i.e Drop column (17.5) -> Followup remove ignore rule (17.6).
1. Due Date: Assign a due date 1 week into the assigned milestone. To see the dates for the milestone, you can click "Preview" after adding the Milestone, then open the milestone link in a new tab, and find its date range at the top of the page.
1. Epic: Assign it to the WebIDE | Technical Debt/Friction or Workspaces Technical Debt Work

epic.

Here's a shortcut of label commands which you can use to easily apply this metadata.

Workspaces:

text
/relate <original issue number or link>
/milestone %"<target milestone>"
/due date <one week into milestone's date, obtained from clicking on milestone link>
/label ~due-date-followup ~refined
/epic &11041

Web IDE:

text
/relate <original issue number or link>
/milestone %"<target milestone>"
/due date <one week into milestone's date, obtained from clicking on milestone link>
/label ~due-date-followup ~refined
/epic &14656

Note that these sorts of issues which we are required to defer
until future releases should not be confused with "tech debt"
work that we are choosing to defer. That is why they use the
following process involving milestones, custom labels, and due date reminders
to ensure that we do not forget to follow up and complete them.

Relationship of Issues to MRs

We want to enforce that:

1. Every MR is owned by a weighted issue

This is in order to facilitate accurate and granular velocity calculations and issue
prioritization under this process.
The merge request is the atomic unit of deliverable work in most cases, so it must be
represented in the prioritization
and calculations by being owned by one and only one issue.

In order to enforce this via triage-ops automations
(/handbook/engineering/devops/dev/create/remote-development/automations-for-remote-development-workflow),
the first line of the issue should have the format: MR: <...>:

1. For new issues, the first description line should be: MR: Pending
1. Once an MR is created for the issue and work is started, the first description line of the issue should be: MR: <MR link with trailing +>,
   and the first description line of the MR should be Issue: <Issue link with trailing +>.
1. If the work for an issue was iteratively split into multiple MR's, the first description line of the issue should be:

   markdown
   MR:
     - <MR link with trailing +>
     - <MR link with trailing +>

   Each description line of the MR's in this list should be Issue: <Issue link with trailing +>. Please note: If breaking out an issue's implementation
   into multiple MR's unexpectedly increases the scope of the work, please consider creating a new weighted and prioritized issue to
   capture the extra scope. This is important in order to accurately reflect scope increases, and their impact on reporting and velocity.
1. If there is NO MR associated with this issue, the first line should be: MR: No MR.
   However, this should be rare, because most issues should have some sort of committed deliverable, even if it is only
   a documentation addition or update. If it is an issue which represents a larger piece of work split across smaller issues,
   then it should be promoted to an epic.

QUESTION: Why does every MR need a backing issue?

- If boards and epics allowed MRs to be added and estimated as well as issues, this would not be necessary - for feature/maintenance
  involving an MR, we could have the MR directly represent the full lifecycle of the discussion and implementation, and not
  have an issue at all.
- We also cannot rely on the Crosslinking Issues feature (https://docs.gitlab.com/ee/user/project/issues/crosslinkingissues.html),
  because this shows ALL linked MRs that have mentioned the issue anywhere, and cannot enforce this 1-1 relationship.

· Handling Issues Outside the Process

Certain group::remote development issues may be categorized under the (workspaces|webide)-workflow::ignored label. These categories include:

1. QA-Owned Issues:
   - Issues owned by QA that may not require the standard Workspaces process.
1. PM-Owned Issues with type::ignore:
   - Issues owned by Product Managers marked with type::ignore, such as reporting, blogs, OKRs,

etc.
1. Long-Lived Security Issues:
   - Security-owned issues with extended timelines that don't align with the typical Workspaces workflow.

This approach ensures that these types of issues do not have an undesired impact on our velocity, and that our Workspaces process remains streamlined while accommodating different issue categories that may not fit the standard workflow.

Wider Board Columns

The default width of lists on boards can make the board harder to use, since you see fewer items and have to scroll more.

There is an open issue to address this. In the meantime, though, you can use the following javascript bookmarklet suggested in this comment on the issue, which will make the lists take up the full board width. Just make a bookmark named "Wider board lists" with this as the link:

text
```
javascript:(function(){var el=document.getElementsByClassName('boards-list');for(i=0;i<el.length;++i){el[i].style.padding=0;el[i].style.display='table';}el=document.getElementsByClassName('board');for(i=0;i<el.length;++i){el[i].style.padding=0;el[i].style.border='0';el[i].style.display='table-cell';}el=document.getElementsByClassName('board-inner');for(i=0;i<el.length;++i){el[i].style.padding=0;el[i].style.border='0';}})();
```

· Communication

The Remote Development Team communicates based on the following guidelines:

1. Always prefer async communication over sync meetings.
1. Don't shy away from arranging a sync call when async is proving inefficient, however always record it to share with team members.
1. By default communicate in the open.
1. All work-related communication in Slack happens in the gcreateide channel.

· Time Off

Team members should add any planned time off in Workday, so that the Engineering Manager can use the proper number of days off during capacity planning.

· Ad-hoc sync calls

We operate using async communication by default. There are times when a sync discussion can be beneficial and we encourage team members to schedule sync calls with the required team members as needed.

· Other Useful Links

· Developer Cheatsheet

Developer Cheatsheet: This is a collection of various tips, tricks, and reminders which may be useful to engineers on (and outside of) the team.

· Fostering Wider Community Contributors

We want to make sure that all the fields of the Create:Remote Development team are approachable for outside contributors.
In this case, if issues should be good for any contribution it should be treated with extra care. Therefore, have a look at this excellent guide written by our own Paul Slaughter!

Cultivating Contributions from the Wider Community: This is a summary of why and how we cultivate contributions from the wider community.

· GitLab Unfiltered Playlist

The Remote Development Group collates all video recordings related to the group and its team members in a playlist in the GitLab Unfiltered YouTube channel.

{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="ide" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="ide" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="ide" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="ide" >}}
{{< /tableau >}}

Create:Remote Development Error Budget

- https://dashboards.gitlab.net/d/stage-groups-detail-ide/stage-groups-ide-group-error-budget-detail?orgId=1
- https://dashboards.gitlab.net/d/stage-groups-ide/stage-groups-ide-group-dashboard?orgId=1

Automations

Automations should be set up via triage-ops if possible.

Other more complex automations may be set up in the
Remote Development Team Automation project.

Automations for Group Workflow

Ideally we should automate as much of the Planning Process workflow as possible.

We have the following automation goals for this Workflow. Unless otherwise noted, these rules are all defined in the triage-ops policies/groups/gitlab-org/ide/remote-development-workflow.yml config files.

ID	Goal	Automation	Link(s) to implementation
01	Warn when no epic is assigned	Issues in ~"Category:(Web IDE | Workspace)" but with no epic assigned should get a warning comment	TODO: implement
02	Assign missing milestone to issues	Issues in ~"Category:(Web IDE | Workspace)" should be assigned to the %"Backlog" if no milestone is assigned	TODO: implement
03	Flag stretch issues in a milestone	Issues that have been assigned the active milestone for example, 16.x, 17.x and don't have a ~refined label and weight should be marked as ~Stretch	TODO: implement
04	Sync Workspace workflow and GitLab workflow labels	Unstarted issues with ~"refined" assigned should get ~"workflow::ready for development" assigned.	TODO: implement
05	Ensure all issues with an assignee have a weight assigned	All refined issues that are not bugs with an assignee but no weight should get a reminder note to either add a weight estimate.	TODO: implement

---
SECTION: engineering/devops/dev/create/remote-development/community-contributions.md

Impact of wider community contributions

GitLab's commitment to fostering a thriving environment for wider community contributions, has been a significant factor in its success and appeal. Contributions from the wider community naturally prevent silos, boost velocity, and encourage engineers to uphold a higher standard of maintainability.

Our Editor Group has a lot to gain from cultivating wider community contributions within our slice of the GitLab product.

State of wider community contributions for the Editor Group

At the time of writing this, less than 2% of all GitLab community contributions target the Editor Group's codebase
(see editor contributions, see all contributions).
For just the devops::create stage, the Editor Group has around 15% of all GitLab community contributions (see create stage).

It appears that the Editor Group has a very small slice of all community contributions. This could be a symptom of a more underlying problem, and/or a missed opportunity.

Problem analysis

> PLEASE NOTE: For this analysis, we assume that GitLab as a whole has a healthy base of wider community contributions. We are only interested in considering issues that arise from the Editor Group specifically.

Here is an interdependency graph breaking down this problem into possible contributing factors and their relationships with possible solutions.

> suggestion: When reading the graph, look for nodes with multiple arrows pointing away. These are areas where a small difference can have a significant spreading effect.

Graph explanations

- Lack of issue visibility: Issues represent the primary Work-To-Be-Done for our organization. If our Editor Group's issues are not visible to the wider community, then naturally we will have low community contributions.
- Lack of contributor motivation: Community members that find Editor Group issues may lack motivation to pick them up. This may be caused by the relevant feature itself (for example, it is niche and doesn't have wide adoption), or simply by the way the issue is triaged and written (for example, it lacks weighting or assumes some prior knowledge).
- Lack of ability to contribute: If a community member is motivated to pick up an issue, they may still be blocked by an inability to make any progress. This can be caused by issues with unclear objectives, unclear implementation steps, and/or a codebase that is hard to understand and modify by newcomers.
- Issues not newcomer friendly: This problem describes situations where issues are focused and written in such a way that cannot be easily understood or picked up by newcomers. This can happen because the issue description has unclear requirements, assumes prior-information, and/or the issue weight is tailored towards specific people.
- Code not newcomer friendly: When our codebase cannot be easily understood and modified by newcomers, then wider community contributions will be naturally discouraged since the Cost-To-Contribute is high.
- Lack of feature adoption: If a feature has low user adoption, then the perceived pay-off for a community contribution can be relatively smaller than the pay-off of other issues.

Symptom of a silo

A codebase that evolves in a silo is difficult to maintain in the long run.

Silos are systems which generate knowledge, but keep that knowledge within themselves. This can create a single-point-of-failure for an organization. It also threatens to deteriorate quality overtime, because the system cannot receive external feedback, nor knowledge foreign to the system.

When there is a healthy stream of contributions from the wider community, the codebase is

naturally protected from growing in a silo. On the other hand, a lack of wider community contributions could be an indicator that the codebase will exhibit other silo-like behavior (most notably, a decline in maintainability). This further prevents outside involvement, creating a negative feedback loop.

Solutions

The following sections describe possible solutions that address the above problems. These solutions are not mutually exclusive and should be prioritized and adopted as needed.

1. Treat wider community as primary audience
2. Improve Hackathon presence
3. Improve code maintainability

Treat wider community as primary audience

The most significant impact towards fostering wider community contributions is to treat the wider community as the primary audience for issues. This directly improves a contributor's motivation for picking up an issue. Issues written this way:

- Are clear and concise (e.g., the most relevant information is kept above the fold and not buried in discussion comments)
- Do not assume prior knowledge (e.g., a code refactoring that references a new pattern by name without any links or details)
- Are weighted and the weight is decoupled from time estimates and specific assignees (see relevant guideline). When issues are weighted for specific individuals, we discourage the wider community from participating and reinforce silos.

With the wider community as the primary audience, the issue writer is forced to consider that someone who picks up an issue may not even know where to get started. This is an incredible motivator to investigate and leave an implementation guide.

What goes into an implementation guide?

This guide can be very brief. It simply contains the result of a informal investigation into related lines of code and possible actions that could resolve the issue. It can also contain steps for breaking up the MR's into iterative chunks.

At the end of the day, the implementation guide guides the implementor, but gives enough flexibility and room for their discretion and creativity.

What if the author isn't equipped to leave an implementation guide?

The issue author does not have to be the one to investigate and leave an implementation guide. Instead, the author should ask someone with domain knowledge for help.

text
@<NAMEHERE>, could you please help me investigate and leave a brief implementation guide for this issue? Thanks!

This seems like a lot of work.

If someone with domain knowledge cannot feasibly and quickly formulate an implementation guide for an issue, then it's a good sign that there might be a requirements gap or even some unique challenges. These unique challenges should be captured in the implementation guide, so that contributors are aware of the entire picture.

This effort of leaving an implementation guide (and specifically the investigation effort) goes along way to informing all stakeholders about the true nature of the issue - from planning, weight estimating, implementing, reviewing, and testing.

Improve Hackathon presence

The GitLab Hackathon is an amazing virtual event of distributed velocity and productivity. It attracts many veteran and first-time contributors.

Participants of the GitLab Hackathon have an incentive to create a lot of MR's during this event, so they are looking for MR's with very clear requirements, a very clear implementation guide, and a low-level of effort. Across GitLab's Frontend, we've had amazing progress made in large project-wide efforts (example, another example) during Hackathons. This is because the relevant issues and epics are geared towards the wider community and leveraging that scale.

How can we improve the Editor Group's issue presence during Hackathon's?

- Write issues and epics with the wider community as the primary audience.
- Look for opportunities to scope issues such that applying the quick win label is appropriate.
- Organize large refactorings into a single epic with clear instructions and an issue for each directory / file (example).

This seems like a lot of work. What do we expect to gain?

The Hackathon attracts new and veteran contributors, and is a great way to mentor people to not only contributing to GitLab, but contributing to the Editor Group's relevant issues. When contributors have a positive experience, they tend to gravitate towards similar experiences (e.g., searching for issues with similar labels or people involved).

An investment in the Hackathon is an investment into fostering a longer-term community.

Improve code maintainability

An issue may be well written and attractive for a new contributor, but contributing to some codebases is simply just difficult. When we think of maintainability, we usually think of our future selves and ask, "Are we able to keep maintaining this"? When it comes to open source software, we should look beyond ourselves and ask, "Would someone completely outside my group be able to keep maintaining this?"

How do we keep a codebase maintainable beyond ourselves?

- Testability. When a change is made, it shouldn't require special knowledge to know if it's

broken something or not - the test suite should fail.
- Modifiability. Is it easy to tweak and make changes to the codebase? Is there high coupling between certain modules? Do single responsibilities live in a single cohesive module?
- Understandability. Does a module make sense to someone with no context? If context is required, is there traceability to that context?

SECTION: engineering/devops/dev/create/remote-development/developer-cheatsheet.md

Useful Commands

general

- Testing CI/CD Jobs Locally: gitlab-runner exec shell jobname

gitlab-org/gitlab

- gdk start
- gdk doctor
- bin/rake frontend:fixtures
- Running tests:
  - yarn karma
  - yarn jest
- Debugging Capybara
  - CHROMEHEADLESS=0 bundle exec rspec spec/features/projects/tree/createdirectoryspec.rb
- Capybara Screenshots
  - screenshotandsavepage
  - screenshotandopenimage
- livedebug
- Static Analysis:
  - scripts/static-analysis (long)
  - yarn eslint (faster)
- fdescribe and fit for focused karma specs

Running QA Specs

See <https://gitlab.com/gitlab-org/gitlab/tree/master/qahow-can-i-use-it> for more details.

- cd qa
- bundle
- brew cask <install|reinstall> chromedriver
- bundle exec bin/qa Test::Instance::All https://0.0.0.0:3000 -- qa/specs/features/ee/browserui/1manage/project/projecttemplatesspec.rb

To run the QA specs in RubyMine, use a custom rspec runner configuration (right click on the arrow next to the example in the gutter), and set the qa/bin/rubymine script as the custom RSpec runner script, and the working directory as qa.

See more detailed instructions for this process here: <https://gitlab.com/gitlab-org/gitlab-

development-kit/-/blob/main/doc/howto/endtoendtestconfiguration.mdstarting-tests-using-the-rubymine-gutter>

gitlab-com/www-gitlab-com

- Run site locally:
  - cd sites/handbook && NOCONTRACTS=true bundle exec middleman
  - (see monorepo docs for more details)
- Development Documentation
  - Testing tips
- You can use this dashboard to select a page or pages and timeframe for page views: https://datastudio.google.com/reporting/e7618539-81b4-4174-9731-3c858e3057b2/page/xXKYB

Tips and Tricks

GDK Tips

- To access EE features, you need to make sure you have an EE license added in /admin/license

Running Web IDE Terminal in GDK

- GDK Docs: Web IDE Terminal HowTo
- YouTube: Web IDE Terminal - Setup using GDK (23:44)

GDK Debugging

Log Debugging

- If you want to print out a debugging message:
  - puts or p will ONLY show up in gdk tail rails
  - logger.info('...') will ONLY show up in tail -f log/development.log

RubyMine Debugging

MOVED: Ths section on debugging the GDK with the RubyMine debugger has been moved to <https://handbook.gitlab.com/handbook/tools-and-tips/editors-and-ides/jetbrains-ides/individual-ides/rubymine/debugging-rails-web>

Git Tips

Rebasing

Interactive Rebasing

- Use of interactive rebasing (git rebase -i master) and message cleanup via reword/fixup are worth learning more about if you are unfamiliar with them.
- Interactive rebasing tip:
  - This process will let you separate "interactive rebasing" against the merge-base (the common ancestor of your branch HEAD and the upstream branch master) from the possibility of "conflict resolution" when rebasing against the latest upstream master.

- First, do a git rebase -i $(git merge-base HEAD master). This lets you do your interactive rebase against the merge-base without any chance of having to deal with conflicts at the same time. Make sure that the merge-base commit is contained on your master branch (i.e. you didn't just fetch and checkout the branch directly without updating master). You could just git fetch then rebase against origin/master, but this negates the benefit of using --force-with-lease.
- Optionally git push --force-with-lease (Or just wait until after the next step to push if you don't want to trigger an extra unnecessary build)
- Then, do a git rebase master, to rebase against the latest master, and resolve any conflicts against your cleaned-up, interactively-rebased branch.
- git push --force-with-lease (force-with-lease ensures nobody else has pushed to the branch since you last pulled)

Using the --onto option for rebasing stacked MRs

If you have multiple MRs which are "stacked" on each other, the --onto option is very useful and can help avoid unnecessary rebase conflicts.

Assume this situation:

1. first MR and branch is based off of the master (main) branch, and has some commits.
1. second MR and branch is based off of the first branch, and has some commits.
1. first branch has been rebased off of master.
1. This means that the first branch's commit's SHAs have changed, and thus the SHAs for those commits in the second branch are outdated.

This is where --onto comes in handy. If you simply tried to rebase second directly on to of first, you would get a lot of confusing rebase conflicts that didn't make sense, because the "history" or state of the first branch has changed compared to what the second branch currently knows about.

But there's a way to avoid these confusing and unnecessary conflicts: You take ONLY the commits that are part of the second branch, and rebase them ONTO the current state of the first branch, using the --onto option.

Here's how:

1. git checkout second
1. git log second and copy the SHA of the commit that USED to be the latest commit on the first branch. For example, abcdef
    - NOTE: If you have ensured that there is only a single commit on the second branch (via regular interactive rebase and fixup as explained above), then you can also just use head^ instead of getting the specific SHA.
1. Rebase "relative" to that commit, "onto" the first branch: git rebase abcdef --onto first, or if second only has one commit, use git rebase head^ --onto first
1. Continue on with the rebase as normal. You may still get some legitimate conflicts you have to resolve if first has changed some of the same lines as second, but no confusing/unnecessary conflicts due to the second branch's "outdated history" of first, because you've ignored it with --onto.

Git References

- See this blog post and our git cheatsheet for more git tips.

Interactive Git Learning Tools

- https://onlywei.github.io/explain-git-with-d3/ is a very cool sandbox site that takes you through tutorials of various git commands, with a real-time visualization of what is going on. You can also type your own commands outside of the tutorial instructions in many cases!
- https://ndpsoftware.com/git-cheatsheet.html is a great reference and visualization of the various git commands grouped into different "areas" in git.

Squashing down a branch which has had master merged into it

When a branch has followed a merge instead of a rebase workflow, it can get very confusing to know what is going on, and you want to just squash it down to a single commit. But, you can't just do a regular interactive rebase relative to master if the branch has had master merged into it. Here's what you have to do instead.

Note there may be more efficient ways of doing this, suggestions are welcome. Also note that sometimes this doesn't work, you'll end up with almost all the changes from the branch uncommitted after step 5 - not sure why :(

Assuming branch is named branch and upstream is named master:

1. Do a log of all the commits on the branch: git log master..branch --oneline
1. Find the last (latest) commit on the branch. It will be the top one, assume it is c60ed83.
1. Find the commit on the branch which is IMMEDIATELY BEFORE the merge commit.
   1. You can run git log --graph --oneline --decorate on your branch to find the merge commit - the merge commit will likely have a commit message like "resoove merge conflicts".
   1. Follow the "line" of your branch down and find the commit on your branch immediately before the merge commit.
   1. Here's an example of what this would look like. In this example, c36ee33 is the merge commit, and b48156a is the commit immediately before it.
      In a real terminal, the lines will also be different colors, to make this easier to read.

      text
      c60ed83 (HEAD -> caw-investigate-rebase-conflicts, origin/caw-investigate-rebase-conflicts) chore: fix vue
      a5269d6 chore: fix vue
      b3941ad chore: fix previous commits
        c36ee33 chore: resolve conflicts
      |\
      | a13f09e (origin/llb/conditionally-load-legacy-web-ide-scripts) Merge branch 'removeRemoteFromExample' into 'main'
      | |\
      | | a5144c2 chore: Remove "Remote Development" mode from the example app
      | |/
      | c33f00c Merge branch 'cwoolley-gitlab-main-patch-d6cd' into 'main'
      | |\
      | | fcfb4a1 (origin/cwoolley-gitlab-main-patch-d6cd, cwoolley-gitlab-main-patch-d6cd)

```
chore: Update file Default.md
      |  |   e97eeec Merge branch 'replaceLogger' into 'main'
      | |\ \
      | |  | 8d48c3e chore: Replace console.log reference with actual logger
      |  | |   60be327 Merge branch 'rename-group-ide-label' into 'main'
      | |\ \ \
      | | ||/
      | |/| |
      | |  | 90ac565 chore: Rename group::ide to "group::remote development"
      | |/ /
      |  |   0b89422 (origin/365-make-vscode-loglevel-configurable-2, origin/365-make-vscode-
loglevel-configurable) Merge branch 'dp-remove-remote-repository' into 'main'
      | |\ \
      | | |/
      | |/|
      | |  fc374f0 feat: Remove "Configure a Remote Connection" command
      | |/
      |    522f9b9 Merge branch 'updateCommit' into 'main'
      | |\
      | |  9a99045 feat: update copy for committing to new branch
      |  |   e6a542f Merge branch 'ealcantara/web-ide-development-process-issue-template' into
'main'
      | |\ \
      | | |/
      | |/|
      | |  a520ee4 (origin/ealcantara/web-ide-development-process-issue-template) chore: Web
IDE development process issue template
       | | b48156a chore: fix waitForReady
       | | 3f4bae3 chore: ran prettier
       | | bc990b1 chore: remove with erroneous configType
       | | 75ad9f0 chore: fix errors
      ...
      ...
```

1. Copy the SHA of that commit which is immediately before the merge commit. For this example, we'll assume its b48156a
1. Reset hard to the commit before your merge commit: git reset --hard b48156a
1. merge --squash the last commit: git merge --squash c60ed83
1. Now, if you run git log --graph --oneline --decorate again, you will see that the merge commit and all the commits after it have been replaced with a single normal, non-merge commit.

Here's an example of what the above log looks like after doing this:

text

```
b48156a (HEAD -> caw-investigate-rebase-conflicts) chore: fix waitForReady
3f4bae3 chore: ran prettier
bc990b1 chore: remove with erroneous configType
75ad9f0 chore: fix errors
...
```

...

1. If there were other merge commits, get rid of them using the same steps.
1. NOTE: In some cases, after the squash, there may be extra changes from the master commits that you don't want. Get rid of them with:
    1. git restore --staged .
    1. git checkout .
    1. git clean -df

Now, you should be able to use rebase and rebase --interactive normally on your branch.

If you want to try this out yourself with the above example, you can check out
this branch
(just don't push it back to the repo).

Working with Issues/MRs

- Expanding Issue/MR Threads for Searching

New habits

Though the contributor and development docs are the single source of truth, there are some additional habits that may be worth developing when you're new to the code contribution process.

Depending on your existing habits and git practices the habits below may help mitigate pain during code submissions.

- Keep GDK up to date (update often, if not daily)
- Keep your commit history clean
  - Take special note of the commit message guidelines
  - See "Git Tips" above
- Keep merge requests small
  - Merge conflicts are inevitable, but focusing on making your MRs smaller will save you pain later
- Keep localization files up to date
  - When adding English copy, messages, or labels don't forget to update localization files

Dealing with Broken Master

- <https://handbook.gitlab.com/handbook/engineering/workflow/broken-master>
- <https://handbook.gitlab.com/handbook/engineering/workflow/merging-during-broken-master>

Browser Testing

- Browser Testing: <https://docs.gitlab.com/ee/development/feguide/index.htmlbrowser-support>
- Dynamic element validation in E2E tests: <https://docs.gitlab.com/ee/development/testingguide/endtoend/dynamicelementvalidation.html>

Links on Testing and Software Design

About testing:

- Vue test utils guide: <https://v1.test-utils.vuejs.org/guides/>
- Book: The way of the web tester: <https://pragprog.com/titles/jrtest/>
- An essay on mocks: <https://martinfowler.com/articles/mocksArentStubs.html>
- Clean architecture book: <https://www.amazon.com/Clean-Architecture-Craftsmans-Software-Structure/dp/0134494164>

Some frontendmasters workshops related to testing that I want to take after the web security workshop:

- <https://frontendmasters.com/courses/testing-practices-principles/>
- <https://frontendmasters.com/courses/testing-javascript/>

JetBrains IDE Usage

MOVED: There is now a dedicated handbook section on JetBrains IDEs: <https://handbook.gitlab.com/handbook/tools-and-tips/editors-and-ides/jetbrains-ides/>

SECTION: engineering/devops/dev/create/remote-development/principles.md

Overview

We outline the Create:Remote Development team principles as practical applications of our CREDIT value system. These principles are not meant to be exhaustive. This page contains our formalized observations about how the Create:Remote Development team operates, and other teams in GitLab may have different perspectives.

Fight for the user

At the core of all our actions, we always fight for our end users. Above anything else, we always start with the desired user experience, then work backward. Our end goal is always to make GitLab the most loveable product it can be, full of features that users want to use.

Question everything

Often in software, it can be tempting to stick to patterns you are familiar with or are biased toward. Biases like these run counter to our goals of developing a lovable product for our users. We aim to keep a mindset of "killing our darlings" or letting go of our dogmas to build better software.

We strive to be bold and ruthless about deleting code or features that don't provide value, or which are too complex (or rarely used) to justify their continued existence.

Embrace challenge and discomfort

Software development is often a challenge, and comes with various forms of discomfort. On the Create:Remote Development team we lean onto this discomfort as an expected part of the development process, and look for creativity in these situations. We quickly accept situations as they are, rather than being upset or worried about them. We don't mind being surprised by challenges during development; we accept them as part of the process.

Keep the fun

Our working life consists of a significant amount of our time. Given that we want to develop the best software possible, on the Create:Remote Development team we embrace having fun as part of our process. When we have fun during our work, we feel trust within our team, and help create an environment conducive to doing our best work.

Often, having fun as part of work is seen as a lack of commitment to your role or the company. In practice, this statement isn't true. In the Create:Remote Development team, we ensure keeping the fun remains part of our culture.

We're all fun people in Create:Remote Development and we embrace that. We strive to bring our whole selves to each meeting and interaction without worry or pause, this is the environment we seek to establish on the team.

Measure what matters

At GitLab, we seek to measure useful data to help us build better software for our end users. In the Create:Remote Development team, we aim to be extremely careful and purposeful about what we choose to measure and to what extent that influences our decisions, based on how much value we feel it offers our end users.

SECTION: engineering/devops/dev/create/source-code/_index.md

Please follow the links below to visit the pages of both engineering teams that make up the Source Code Management group.

SECTION: engineering/devops/dev/create/source-code/backend/index.md

The Create:Source Code BE team focuses on GitLab's Source Code Management (SCM) tools and is responsible for all backend aspects of the Source Code group's product categories in the Create stage of the DevOps lifecycle. For information on our product direction, visit the Category Direction - Source Code Management page.

We interface with the Gitaly and Code Review teams, and work closely with the Create:Source Code Frontend team. The features we work with are listed on the Features by Group Page and technical documentation is available on the Create: Source Code Backend page.

About our team handbook page

This is our central document for finding everything important to our team. It is our single source of truth for who is on the team, processes, practices, meetings, links, channels, metrics and more. On this page, a team member should be able to find everything they need to be fully engaged on this team.

Updating this page

To be a DRI for updating our team handbook page, consider following these steps:

- Navigate to our handbook update epic.
- Create a sub issue describing why a change to this page is needed.
- If it's quick and you have context, weight it as 1 and create an MR with the change.
- If it's going to require more effort, weight it higher and describe what's needed for a successful change. It can be considered during the next milestone planning.
- Once the MR is ready, mention @gitlab-com/create-team/source-code/backend in a comment asking for feedback. Mentioning the whole team ensures everyone on the team can contribute to how the team operates.
- If you think this might be an opportunity to share documentation cross functionally, consider pinging the frontend team to get their feedback.
- Assign the EM as the reviewer.
- Once a the team has had 2 business days to discuss, and any concerns are resolved, follow up and ask the EM if they can merge.

Team members

The following people are permanent members of the Create:Source Code BE Team:

{{< team-by-manager-role role="Senior Engineering Manager(.)Create:Source Code" team=".(Backend).Create:Source Code" >}}

Stable counterparts

The following people of other functional teams are our stable counterparts:

{{< engineering/stable-counterparts role="(Product Manager|Frontend Engineer|Technical Writer|Software Engineer in Test|Senior Security Engineer).(Create:Source Code|Create \(Source)|Dev\:Create" >}}

Common Links

- GitLab Team Handle: @gitlab-com/create-team/source-code/backend
- Slack Channels: gcreatesource-code-be, gcreatesource-codestand-up, gcreatesource-code, screate
- Team error budget - Group Dashboard
- Team error budget - Detail Dashboard

Sisense and KPIs

To help us stay on track with Development KPIs, we use a metrics dashboard. This dashboard includes security MRs from production, but doesn't include security MRs from dev.gitlab.org. For team-specific data and metrics, ensure you filter by our team.

{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="source code" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="source code" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="source code" >}}
{{< /tableau >}}

{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="source code" >}}
{{< /tableau >}}

Workflow

We use the standard GitLab engineering workflow. To create an issue for the Create:Source Code BE team, add these labels:

- ~backend
- ~"devops::create"
- ~"Category:Source Code Management"
- ~"group::source code"

For more urgent items, use the gcreatesourcecode Slack channel.

Take a look at the features we support per category here.

Working with the Product Team

Weekly calls between the Product Manager and Engineering Managers (frontend and backend) are listed in the "Source Code Group" calendar. Everyone is welcome to join and these calls are used to discuss any roadblocks, concerns, status updates, deliverables, or other thoughts that impact the group.

Issue refinement

1. Once we have validated the problem, product, UX, and engineering will collaborate to propose

a solution and decide on what's technically feasible. The proposed solution may be shared with users to validate it solves the problem.
    1. Issues that require design work are marked with UX and workflow::ready for design.
    1. Issues in the design process are marked with workflow::design.
    1. Once designs are ready and the proposed solution is viable then the label workflow::planning breakdown will be applied.
1. Once we have confirmed the proposed solution is viable, we will move to break it down as much as possible. When issues are ready for this stage, PM will mark issues with workflow::refinement label to signal next step.
1. EM will create a refinement issue (example) and distribute tasks labeled workflow::refinement among engineers.
1. Engineers or EM will follow the checklist for assigned issues, work with PM, UX, and other engineering counterparts where necessary to address questions and concerns.
1. If the planned implementation of the issue can be further broken down, the engineer/EM will work with the PM to reduce scope and create new issues until this is the case (either PM or engineer/EM can create new work items).
1. Once an issue is fully refined, engineers or EM will add an appropriate weight and label it as workflow::ready for development. These issues can then be added to the milestone.
1. When other teams depend on Source Code Backend issues planned for the current milestone, those issues will be labeled as SCM::AwaitingBackend

Note: if an issue receives a weight > 3 after this process, it may indicate the IC may not have a full idea of what is needed and further research is needed.

Diagram

mermaid
flowchart TD
    Start([When the issue is ready for engineering review])

    subgraph PM[Product Manager]
        RefinementLabel[Apply workflow::refinement label]
        click RefinementLabel "https://gitlab.com/groups/gitlab-org/-/issues/?sort=createddate&state=opened&labelname%5B%5D=group%3A%3Asource%20code&labelname=workflow%3A%3Arefinement&labelname%5B%5D=scm-backlog&firstpagesize=20" blank
    end

    subgraph EM[Engineering Manager]
        CreateRefIssue[Create refinement issue]
        DistributeTasks[Distribute tasks to engineers]
    end

    subgraph ENG[Engineer/EM]
        RefineIssue[Follow refinement checklist]
        NeedBreakdown{Can be broken down?}
        CreateNewIssues[Create smaller issues]
        FullyRefined[Issue fully refined]
        ReadyLabel[Apply workflow::read